

FIDEM: A Standard-Compliant Framework for Secure Binding of MUD Profiles to IoT Devices

Alessandro Lotto

University of Padua
Padua, Italy

alessandro.lotto@math.unipd.it

Savio Sciancalepore

Eindhoven University of Technology
Eindhoven, Netherlands

s.sciancalepore@tue.nl

Alessandro Brighente

University of Padua
Padua, Italy

alessandro.brighente@unipd.it

Mauro Conti

University of Padua
Padua, Italy

mauro.conti@unipd.it

Abstract—The Manufacturer Usage Description (MUD) standard enables enforcement of network restrictions for IoT devices based on their expected network traffic, as specified by manufacturers in an online MUD file. Devices advertise a URL pointing to this file, yet the standard does not define how to securely bind the issuing device to its profile. As a result, malicious devices can manipulate network policy enforcement by advertising valid URLs referencing genuine MUD profiles, but not intended for that device. Although MUD defines a certificate-based secure issuance method, current deployments rely on the insecure DHCP-based extension due to simpler integration. Existing solutions either depend on Public Key Infrastructure (PKI), break standard compliance, require excessive active manufacturer involvement, or overlook secure profile updates. In this paper, we present FIDEM, a standard-compliant framework for securing DHCP-based MUD URL issuance. FIDEM provides cryptographic binding between IoT devices and their MUD profiles by leveraging Zero-Knowledge-Proof authentication, eliminating PKI reliance, minimizing manufacturers' involvement, and supporting secure profile updates. Formal analysis shows that FIDEM withstands stronger adversaries than in prior work, including supply-chain compromise and attacks using legitimate devices as cryptographic oracles. Our real-world evaluation on two reference constrained devices (ESP32-S3 and ESP32-C6) demonstrates minimal overhead compared to standard DHCP ($\sim 5ms$ and $\sim 20mJ$) and significant improvements over certificate-based benchmarks ($\sim \times 20$ faster, and $\sim 35\%$ less energy).

Index Terms—Manufacturer Usage Description (MUD), IoT Security, Secure Binding, Zero-Knowledge Proof.

1. Introduction

The increasing integration of Internet-of-Things (IoT) devices into cyber-physical systems is enhancing automation, process optimization, and safety [1], [2]. However, the constraints of such devices and networks make it challenging to deploy advanced resource-consuming security mechanisms [3]–[6]. Thus, IoT devices represent attractive targets for attackers seeking to exploit weak protections to

access sensitive data or launch distributed attacks [7]–[9]. To mitigate this expanded attack surface, the Internet Engineering Task Force (IETF) recently introduced the Manufacturer Usage Description (MUD) standard [10]. MUD enables manufacturers to disclose the expected network behavior required for the proper functioning of their IoT devices, i.e., the *MUD profile* [11]. These profiles are formalized into *MUD files*, which are signed by the manufacturer and stored on a *MUD Server*. A dedicated *MUD Controller* to be deployed on the user side can retrieve the MUD file referenced by the *MUD URL* issued by the IoT device at network joining time. Then, the Controller can translate the MUD file into network policies and restrictions where any behavior not described in the file is considered unauthorized [10]–[12].

Despite growing interest, several open problems in management, security, and implementation limit the real-world adoption of MUD today [11]–[15]. A critical concern is the Device-to-MUD Profile Binding (*MUD-Binding*) vulnerability, which affects the security of the standard Dynamic Host Configuration Protocol (DHCP)-based URL issuance procedure [16], [17]. Given the public nature of MUD URLs, a compromised device can issue a spoofed URL that points to an incorrect, yet authentic, MUD profile, thereby leading to the enforcement of incorrect network restrictions [12], [16]. The key factor enabling such attacks is that MUD decouples policy retrieval from device authentication. The Controller accepts a MUD URL based on the authenticity of the retrieved file rather than any proof that the requesting device is entitled to use that profile [10], [13]. Thus, a spoofed MUD URL does not inherently raise inconsistencies detectable by the Controller, and any device capable of advertising a valid URL can trigger the installation of (possibly incorrect) corresponding policies. An extension based on X.509 is available for secure URL issuance [10]. However, it is not supported in practice and current MUD implementations rather rely on the insecure DHCP extension, due to more convenient plug-and-play device installation [11], [13], [18]–[21]. At the same time, symmetric-key alternatives raise security concerns. The Controller would require access to device-specific secret keys, but an attacker could set up a malicious Controller to recover secret keys of target devices and reuse them to

impersonate other devices. Furthermore, the solutions proposed in the literature present additional limitations, such as lack of standard compliance and secure MUD profile update mechanisms and excessive manufacturer involvement during *MUD-Binding* verification. Such operational challenges make solving the *MUD-Binding* vulnerability particularly challenging, and motivate the following Research Question (RQ) driving this work:

RQ: How can we enforce cryptographically secure *MUD-Binding*, without relying on both a PKI and on manufacturer involvement during the verification procedure?

Contribution. In this paper, we introduce FIDEM, a standard-compliant framework for secure DHCP-based MUD URL issuance. FIDEM establishes a cryptographic binding between the issuing device and the retrieved MUD profile leveraging Schnorr-based Zero-Knowledge Proof (ZKP) authentication, without relying on PKI and minimizing manufacturer involvement. We design FIDEM to withstand a stronger, yet realistic threat model compared to prior work, explicitly capturing supply chain compromise scenarios and adversaries capable of exploiting legitimate devices as cryptographic oracles in place of compromised devices. Furthermore, FIDEM also introduces a unique formal MUD Management Model (*MMM*), which explicitly specifies how manufacturers could efficiently maintain and associate MUD profiles within the system model. We formally evaluate FIDEM security properties with ProVerif and evaluate its performance against baseline HMAC- and certificate-based approaches in a Proof-of-Concept (PoC) implementation with commercial IoT devices (ESP32-S3 and ESP32-C6). Our results show minimal overhead compared to the insecure DHCP extension ($\sim 7\%$, $5ms$), and significant savings compared to certificate-based approaches, i.e., FIDEM is up to $\times 20$ times faster and $\sim 35\%$ less energy-consuming compared to the X.509 extension with TLS. In summary, this paper makes the following contributions:

- We formalize a stronger yet realistic threat model for MUD than prior work, which also considers supply-chain compromise and adversaries capable of exploiting legitimate devices as cryptographic oracles.
- We present FIDEM, a standard-compliant framework that enforces cryptographic binding between IoT devices and MUD profiles via lightweight ZKP. Compared to prior work, FIDEM avoids PKI, minimizes manufacturer involvement, and supports secure profile updates.
- We introduce a unique class-based *MMM* enabling scalable profile association, efficient key management, and lifecycle support.
- We implement FIDEM on real IoT devices, compare it to state-of-the-art benchmark approaches, and make the source code available open source¹ (anonymized for submission).

1. <https://github.com/aleLtt/FIDEM>

Roadmap. The remainder of this paper is organized as follows. Sec. 2 introduces the MUD standard. Sec. 3 reviews vulnerabilities and limitations in existing solutions. Sec. 4 outlines our system and threat model. Sec. 5 provides the details of FIDEM. Sec. 6 presents a discussion on its security guarantees. Sec. 7 reports experimental results. Sec. 8 discusses limitations and future directions. Finally, Sec. 9 concludes the paper.

2. Preliminaries on MUD

MUD grounds on the observation that most IoT devices have a narrow operational scope and, therefore, require only a restricted set of network interactions [12]. To enforce this principle, manufacturers formally define the expected network behavior of their IoT devices through a set of Access Control Lists (ACLs). These specify the minimal set of necessary and allowed network communications for the correct functioning of the IoT device [21]. The collection of the ACLs constitutes the device’s MUD profile, which is expressed in a *MUD file* using Yet Another Next Generation (YANG) modules and serialized in JSON format [10], [22]. Thus, the MUD file serves as a formal declaration of the device’s native operational requirements.

Architecture and File Retrieval. Fig. 1 illustrates the MUD architecture and file retrieval procedure [10]. The process begins when the IoT device issues a *MUD Universal Resource Locator (URL)* to the network router (step 1), which identifies the location of the device’s MUD file. The router forwards the URL to the MUD Controller (step 2), which in turn retrieves the MUD file from the MUD Server via HTTPS (step 3). Before applying the rules, the MUD Controller verifies the profile authenticity using the associated Cryptographic Message Syntax (CMS) object (step 4). Once validated, the MUD Controller translates the ACLs specified in the file into enforceable network policies, which the router applies to restrict the device’s communications. According to the standard, the MUD profile remains valid as long as the device is connected to the network [10]. In some deployment scenarios, such as smart homes, the MUD Controller logic can be integrated directly into the router, avoiding URL relaying [12], [19], [22].

URL Issuance. The MUD standard defines three extension for URL issuance. [10]. The X.509 extension introduces a non-critical certificate field compliant with IEEE 802.1AR, that contains a single URL pointing to the online MUD file. In this case, the legitimacy of the MUD URL relies on the authenticity of the certificate used for device authentication. Conversely, the DHCP and Link Layer Discovery Protocol (LLDP) extensions embed the URL into a string `MUDstring = MUDurl, [", "], reserved`, where `MUDurl` is an HTTP-based URL compliant with RFC7230 [23], and `reserved` is reserved for future extensions. Unlike X.509, DHCP and LLDP do not provide any built-in mechanism to authenticate the transmitted URL and ensure *MUD-Binding*. This lack of protection introduces security issues, such as spoofed MUD advertisements, which can lead to incorrect enforcement of overly permissive

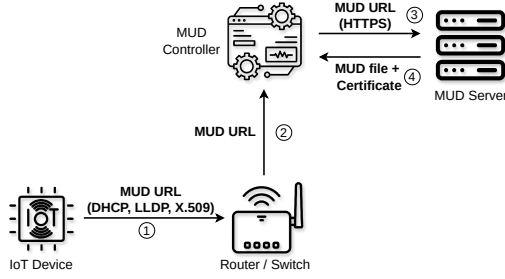


Figure 1: MUD architecture and file retrieval procedure [10].

profiles. Such misconfigurations can expose the network to compromise, enable lateral movement, and facilitate privilege escalation attacks [12], [17].

3. Related Work

The MUD URL issuance mechanism for MUD retrieval is the cornerstone of the MUD, determining which set of network policies (i.e., MUD file) the Controller should enforce based on the received URL. DHCP and LLDP extensions lack security guarantees to ensure the integrity of the issued URL [10], [11]. To the best of our knowledge, no prior work focuses on securing these extensions, although DHCP is extensively considered in MUD implementations and other works using MUD [12], [19], [21], [24], [25]. Table 1 summarizes existing approaches specifically designed to secure the MUD URL issuance mechanism.

Device Authentication vs. MUD-Binding. Solutions such as eMUD [22], MUDscan [26], and MUDThread [8] authenticate devices before retrieving the MUD file. However, eMUD is vulnerable to replay attacks as it relies on static Digital Certificates (DCs) with no challenge-response interaction. MUDscan introduces challenge-response authentication, but using the manufacturer’s public key. Thus, it mainly ensures that the device belongs to the specific manufacturer, rather than providing *MUD-Binding*. Similarly, MUDThread and MeshGuard leverage IEEE 802.15.4 link layer encryption and authentication [27], [31], but do not ensure *MUD-Binding*. Lastly, proposals from Jimenez J. [29] and Richardson *et al.* [30] do not provide any cryptographic mechanism to protect the issued URL. These examples show that *MUD-Binding* is a distinct security property than device authentication. A compromised device can first authenticate using legitimate credentials, then issue a spoofed MUD URL. Thus, *MUD-Binding* is a challenging security property to ensure beyond traditional device authentication.

Manufacturer-Involved Approaches. Garcia *et al.* [16] and Matheu *et al.* [28] adopt a different strategy for MUD file retrieval: the MUD Controller receives the URL directly from the manufacturer after successful device authentication. While this strategy guarantees binding (assuming correct authentication), it introduces significant operational overhead for manufacturers. In fact, they must maintain live authentication services, perform cryptographic operations,

and handle additional traffic for MUD purposes. We argue that such a workload primarily serves end users and rather discourages manufacturers from adopting MUD, thus limiting its widespread adoption. To promote adoption, any solution for secure *MUD-Binding* should minimize manufacturers involvement beyond generating, updating, and serving MUD files.

PKI Limitations. A proposal from Richardson *et al.* [17] offers secure binding and is fully compliant with the MUD standard, without requiring active manufacturer involvement during MUD file retrieval. However, it relies on X.509 certificates and a PKI, which have seen limited support in practice. Although RFC 8520 defines a non-critical X.509 extension, to the best of our knowledge, no available or open implementations currently support it [11], [13], [18]–[21]. Furthermore, native X.509/802.1AR support remains limited in consumer IoT devices due to operational issues related to the certificate size, computational overhead, and lifecycle management challenges for constrained devices [4], [5], [16], [28], [32]. This is also highlighted by the fact that the majority of related works avoid relying on PKI (see Table 1). Recent efforts, such as C509 certificates, aim to provide compact encodings for constrained environments [33]. However, these are still in draft status and have not yet been standardized. By contrast, DHCP remains the *de facto* baseline for IP address assignment in IoT [34], driving industry preference for DHCP extension over X.509. This trend creates a deployment-driven motivation to focus on strengthening the DHCP extension.

Standard Compliance. None of the previous works, except [17], is standard-compliant (see Table 1). Breaking the standard compliance for MUD file retrieval creates interoperability issues and increases the complexity of management and enforcement. In contrast, a standard-compliant solution is highly desirable, as it favors interoperability and adoption.

Secure MUD Profile Update. A secure MUD profile update mechanism is crucial to maintain the integrity and consistency of enforced policies over time. Without it, attackers could exploit outdated or tampered profiles, potentially bypassing security restrictions. Despite its importance, existing solutions except Richardson *et al.* [17] do not address this aspect, leaving a remarkable gap in the overall security of the MUD framework.

Alternative Approaches. Alternative approaches, such as remote attestation or certificateless encryption, may seem viable solutions to ensure *MUD-Binding*. Remote attestation verifies that the device is in a genuine state [35]–[37], which could indirectly support trust in the issued URL. However, its scope is broader than *MUD-Binding*, as it focuses on device integrity rather than the correctness of specific configuration data. Moreover, secure attestation typically requires expensive hardware roots-of-trust and computational energy [35], [37]. On the other hand, certificateless encryption aims to reduce dependence on traditional DCs. However, it still relies on a trusted key generation center and introduces key escrow concerns [38]–[40]. These limitations make both approaches impractical for lightweight, standard-compliant MUD deployments, where simplicity and interoper-

TABLE 1: Comparison of existing approaches for MUD file retrieval and MUD-Binding verification.

Method	URL Issuing Method	Cryptographic Approach	MUD std Compliance	Avoid PKI	Define MMM	Avoid Manufacturer Involvement	Secure URL Update	MUD-Binding
<i>eMUD</i> [22]	Not specified	Certificate-based Authentication	X	X	X	X	X	X
<i>MUDscan</i> [26]	Not specified	Public-Key challenge-response	✓*	X	X	X	X	X
<i>MUDThread</i> [8]	Mesh Link Establishment Parent Request	AES-128 CCM (IEEE 802.15.4 link layer)	X	✓	X	✓	X	X
<i>MeshGuard</i> [27]	Mesh Link Establishment Parent Request	AES-128 CCM (IEEE 802.15.4 link layer)	X	✓	X	✓	X	X
<i>Garcia et al.</i> [16]	PANA	EAP-based authentication	X	✓	X	X	X	✓
<i>Matheu et al.</i> [28]	CoAP	EAP-based authentication	X	✓	X	X	X	✓
<i>Richardson et al.</i> [17]	X.509	Certificate-based Authentication	✓	X	X	✓	✓**	✓**
<i>Jimenez J.</i> [29]	CoAP	N.A.	X	✓	X	✓	X	X
<i>Richardson et al.</i> [30]	QR-codes	N.A.	X	✓	X	✓	X	X
<i>FIDEM</i>	DHCP	EC-based ZKP authentication	✓	✓	✓	✓	✓	✓

* No specific URL issuing method is provided. However, we can reduce to one of the standard approaches.

** *MUD-Binding* assurance for profile updates relies on manufacturer's secure URLs management.

erability are key factors.

In summary, existing solutions fail to guarantee the *MUD-Binding* property, impose excessive operational overhead on manufacturers, or lack standard compliance. Furthermore, existing proposals overlook secure profile updates, leaving devices vulnerable to outdated or tampered configurations. Alternative approaches, such as remote attestation or certificateless encryption, introduce complexity and hardware requirements incompatible with constrained environments. These gaps highlight the need for a lightweight, standard-compliant mechanism that enforces *MUD-Binding* without relying on heavy cryptographic infrastructures or requiring active manufacturer involvement.

4. System and Threat Model

System Model. Fig. 2 illustrates the system model considered in this work. We consider a local MUD-enabled network composed of heterogeneous resource-constrained IoT devices, produced by various manufacturers and connected to a central router. Devices issue their MUD URL using the available DHCP extension. We consider a local IoT network composed of heterogeneous IoT devices, produced by various manufacturers and connected to a central router. Following common practice, we integrate the MUD Controller into the router [12], [19]. To clearly delineate the scope of this work, we distinguish between the *MUD Domain* and the *Operational Domain*. The former includes the MUD Controller, the MUD Server, and all traffic and operations governed by the MUD framework. Conversely, the latter encompasses all existing infrastructure, services, and procedures required for device functionality, independent of MUD. Within the *Operational Domain*, we model an *Operating Server* that handles routine device operations such as updates, authentication, and management tasks. These are performed using pre-existing mechanisms and are outside the scope of MUD and this work. By contrast and according to the limitations identified in Sec. 3, we assume that the MUD Server is a passive entity whose sole role is to serve the MUD file corresponding to a given URL. It does not participate in cryptographic operations related

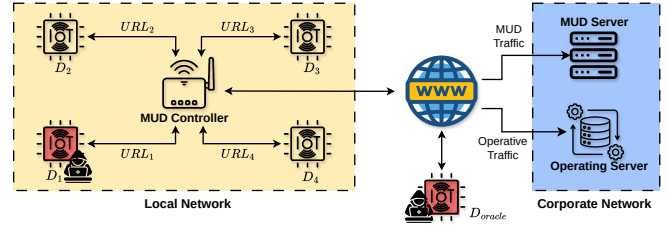


Figure 2: System and threat model. D_1 is compromised, and the attacker may use legitimate device(s) D_{oracle} as a cryptographic oracle to perform operations on behalf of D_1 .

to authentication and *MUD-Binding* verification. Finally, we adopt a standard cryptographic model in which secret material stored on devices is protected in secure hardware. Although we acknowledge that some IoT devices lack strong hardware protections, this assumption is consistent with a large body of prior work [26], [41]–[44].

Threat Model. We consider an advanced, yet realistic and conservative threat model, drawing on examples and motivations from real life. We consider that IoT devices cannot be assumed trustworthy at the time of deployment, as they can be compromised at any stage of their lifecycle, including manufacturing or distribution, reflecting well-documented exposure of IoT supply chains [45]–[47]. We thus focus on two representative compromise scenarios, namely *Compromised Legitimate Device* and *Counterfeit Device Impersonation*, both leveraging binding limitations of the MUD standard architecture [10]. In the former, the attacker compromises a genuine device and aims to induce the enforcement of a more permissive network policy than the one originally intended for that device. Since MUD URLs are publicly accessible, an attacker can systematically analyze the product ecosystem of a manufacturer identifying those with more permissive policies. Compromised devices thus advertise spoofed URLs corresponding to other devices from the same manufacturer, whose MUD profile grants broader communication privileges. This behavior is consistent with prior observations in the literature [22], [26]. In the latter scenario, the attacker introduces counterfeit

devices into the system, presenting them as legitimate products of a target manufacturer. Although this may appear an overly strong adversary, it is in fact supported by real-world incidents involving fraudulent networking and IoT equipment commercialized through third-party vendors or online marketplaces [45]–[47]. In such a scenario, since the attacker does not need to break cryptographic protections or impersonate manufacturer signatures, counterfeit devices can simply advertise a valid URL associated with a legitimate MUD file. Device-level authentication, if any, typically occurs at later stages (e.g., during Cloud-based onboarding), and it does not prevent such attack, leading to the enforcement of (incorrect) network policies based on the spoofed URL [13], [16], [28]. In line with our system model, we consider an attacker that cannot directly access secret material stored on devices or the manufacturer’s private keys. However, the attacker can access to one or more legitimate devices of the target manufacturer. This reflects a realistic capability, as such devices are commercially available and can be freely acquired. Thus, the attacker can leverage them as cryptographic oracles, enabling cryptographic operations without possessing the underlying secrets. We model the set of all such devices as a logical entity D_{oracle} , which may be locally or remotely accessible. Communication between a compromised device D_1 and D_{oracle} is assumed to occur through the network infrastructure, in particular via the router enforcing MUD policies. This interaction can be facilitated by permissive rules commonly observed in MUD profiles, such as those allowing communication among devices from the same manufacturer [12], [13], [26]. These rules make such communication appear as legitimate from the network perspective, further complicating detection. We explicitly exclude out-of-band channels (e.g., Bluetooth, cellular) from the model, to maintain focus on network-enforced policies.

Attacker objectives. Within this threat model, the attacker aims to achieve three objectives: (i) recover secret material through passive observation of communications (**O1**), mislead the Controller into classifying a device as belonging to a different manufacturer (**O2**), and induce the enforcement of an unintended MUD profile within the same manufacturer domain (**O3**). The latter captures the core *MUD-Binding* vulnerability, namely, the ability of an attacker to exploit the lack of binding between devices and their declared MUD profile. We analyze the security of FIDEM against these objectives in Sec. 6.

5. FIDEM

This section presents FIDEM, our framework designed to enforce cryptographic *MUD-Binding*. First, Sec. 5.2 introduces the concept of *MMM*, and Sec. 5.3 presents our proposed MUD file extension to enable the MUD Controller to retrieve the necessary cryptographic material to carry out *MUD-Binding* verification. Then, in Sec. 5.4 we detail FIDEM ZKP-based verification procedure, including parameter embedding and profile update. Table 3 (Appendix A) summarizes the terminology used in this work.

TABLE 2: FIDEM design requirements.

#	Design Requirements
R1	Avoid PKI dependence
R2	Standard compliance and backward compatibility
R3	Prevent MUD URL redirection attacks
R4	Minimize manufacturer involvement
R5	Enable secure MUD profile updates

5.1. Design Requirements

FIDEM relies on five main design requirements, summarized in Table 2. Although PKIs provide a natural way to bind identities and resources, setting up and maintaining such an infrastructure introduces considerable complexity and management costs [48], [49]. Therefore, although an extension for MUD based on X.509 is available for secure URL issuance [10], current MUD implementations rather rely on the insecure DHCP extension [11], [13], [16], [18]–[21]. These considerations motivate our design requirement of *avoiding reliance on PKI* (**R1**), and rather rely on the plug-and-play DHCP extension. This consideration creates a deployment-driven need to secure the DHCP extension for URL issuance. However, several security and implementation challenges arise when establishing secure DHCP-based *MUD-Binding* against our advanced threat model (Sec. 4) without relying on a PKI infrastructure. A major limitation of the solutions proposed in the literature is their lack of standard compliance. Adopting non-standard compliant solutions significantly reduces interoperability and increases implementation complexity. This particularly holds for the Controller, which would need to support multiple non-standard URL issuance mechanisms. Therefore, any security mechanism and MUD extension should fit the standard by design, seamlessly integrating with the existing specification to increase interoperability and avoid increasing complexity. This directly motivates the need for *standards compliance and backward compatibility* (**R2**), ensuring that the proposed solution can be incrementally deployed without disrupting existing infrastructures. Another challenge arises when considering symmetric-key approaches. At first glance, mechanisms based on shared secrets (e.g., HMAC) appear attractive due to their efficiency. However, the MUD Controller would need access to device-specific secret keys, which require secure key provisioning from manufacturers. Besides increasing operational costs due to secure key provisioning, such a strategy introduces critical security issues. An attacker could legitimately set up a malicious Controller to recover secret keys of target devices and reuse them to impersonate other devices. Preventing such misuse leads to the requirement of *resistance to MUD URL redirection and impersonation attacks* (**R3**). Moreover, an overlooked implementation aspect in the literature involves the role of manufacturers in the MUD ecosystem. Existing approaches heavily rely on manufacturer-assisted authentication, where devices or controllers interact with manufacturer-operated services at runtime. However, manufacturers of IoT devices

typically produce and maintain tens or even hundreds of products. Thus, it would be extremely expensive for such manufacturers to maintain an additional always-on infrastructures that actively responds to authentication requests at every device deployment. We argue that such operational costs would ultimately discourage the adoption of MUD in their products. Therefore, a key challenge is to design a solution that enforces secure *MUD-Binding* without requiring active manufacturer participation, motivating the requirement of *minimizing manufacturer involvement* (R4) and limiting them to hosting and maintaining information on a public web page rather than actively interacting with on-line parties. Finally, a further challenge lies in the dynamic nature of IoT devices. MUD profiles could change over the life of a device, e.g., due to a software update, firmware upgrade or inadvertent mistakes [11], [17]. For example, a smart camera may receive a firmware update with new functionalities requiring it to contact new vendor analytics servers that were not part of its original MUD profile. Ensuring that updates are correctly propagated and securely enforced considering aforementioned challenges is a non-trivial problem. This leads to the requirement of *supporting secure MUD profile updates* (R5).

5.2. MUD Management Model

An efficient MUD management strategy is essential to balance security, system complexity, updates, and overhead related to keys, MUD files, and certificate storage. Despite its importance, existing works focus on the technical implementation of MUD-based solutions, overlooking how manufacturers are expected to manage their MUD files in practice. Assigning a unique MUD file and URL to each device, although technically feasible, is inefficient. Devices with the same network behavior would unnecessarily be associated with distinct files and URLs, resulting in an increased management and signing overhead. At the same time, we observe that MUD files tend to be oriented toward classes of devices rather than individual instances [10], [29]. Building on this observation, we propose a *MMM* in which devices are grouped into classes, each assigned a single MUD profile. Devices within the same class share identical network requirements and are therefore linked to the same MUD file and URL. For instance, a smart camera and a smart doorbell from the same manufacturer, although having different purposes, may require the same network services, thus having the same network profile. This model significantly reduces management and storage overhead, as the number of MUD files to sign and store scales with the number of distinct MUD profiles, rather than with the total number of IoT devices.

Definition 1 (MUD Class). A MUD Class is a group of devices that share the same network requirements for proper functioning. Each MUD class corresponds to a single MUD profile and is associated with one MUD file. A MUD class C , defined within a manufacturer M , is uniquely identified by $MUD_{ID} = [M, C]$.

Manufacturers are uniquely identified using standardized identifiers, such as the Organization Unique Identifier (OUI) or Extended Unique Identifier (EUI) [50]. Different firmware versions may result in different network profiles, leading to distinct MUD files classes. Conversely, different firmware versions that however exhibit the same network behavior belong to the same MUD class.

MUD File Update. During a device’s operational life-cycle, software updates may alter its network behavior [11], [17], thereby requiring a change in the associated MUD profile and reassignment to a different MUD class. In this case, the device must receive the new MUD URL as part of the update content. This triggers a URL reissuance process, in which the MUD Controller obtains the new MUD file and updates the device’s network policy accordingly. Additionally, MUD files may occasionally contain errors or incomplete information, which requires corrections or replacements [17]. In this second case, the device and the MUD Controller are typically unaware of the change [17]. To address this, we adopt a publish-subscribe model [13]: the MUD Controller stores the signature of the current MUD file and periodically checks for updates. A change in the signature indicates a file update, triggering the MUD Controller to retrieve and validate the new file, updating network policies accordingly.

5.3. Key Management and MUD File Extension

We propose a key management approach that associates a secret key K_c with each MUD class C . Every manufacturer provides IoT devices with the secret key corresponding to their MUD class, namely K_c . According to our system model, we consider this key to be stored in secure hardware, ensuring protection against physical extraction. For each class, we also define public cryptographic parameters: an Elliptic Curve (EC), a base point G of prime order n , and the point $X_c = K_c \cdot G$ on the curve. The point X_c binds the secret key K_c to the MUD class C , enabling verifiable association without exposing the secret. We thus propose extending the standard MUD file template with a cryptographic module containing these parameters together with MUD_{ID} . We provide the extended MUD file template in Appendix B. In this extension, `manufacturer-id` and `class-id` represent the manufacturer identifier and the MUD class identifier, respectively; `curve` specifies the elliptic curve name, which determines the base point G ; `p-format` indicates whether the elliptic curve point X_c is given in compressed or uncompressed format; `class-key` contains the public point X_c ; `hash-alg` defines the hashing algorithm used during *MUD-Binding* verification. Upon signature verification, the Controller extracts this cryptographic information to carry out the *MUD-Binding* procedure. This approach is efficient since it leverages the MUD class abstraction, reducing the number of keys to manage from per-device to per-class.

5.4. MUD Binding Verification

Binding Verification Problem. Ensuring *MUD-Binding* under the defined design constraints and *MMM* requires an IoT device to prove to the Controller that it possesses the secret key K_c associated with the public value X_c specified in the MUD file. At the same time, such a proof must not reveal K_c to the Controller, and neither require its direct disclosure or transfer. This requirement is fundamental to prevent key leakage, preserve security under the considered threat model, and avoid the limitations of symmetric-key approaches. ZKP schemes provide a well-suited cryptographic primitive to address this mentioned challenge, allowing us to design a scheme to enforce *MUD-Binding* while satisfying the constraints of standard compliance, scalability, and minimal manufacturer involvement.

Binding Verification Mechanism. During manufacturing, the IoT device is equipped with the MUD URL associated to the MUD class it belongs to, the corresponding secret key K_c , and the MUD class public parameters X_c , EC , G , n . At network joining time, the IoT device has to securely deliver its MUD URL, proving the *MUD-Binding* property. To this end, it performs the *MUD-Binding* verification procedure shown in Fig. 3 and discussed below.

- 1) The IoT device samples a random value $r \sim \mathcal{U}(n)$ and stores it in secure memory. Then, it computes the EC point $R = r \cdot G$ as commitment for the sampled r . It then sends a DHCP *Discovery* message including option 161 (see Sec. 5.4.1), which carries the MUD URL and the commitment R .
- 2) The MUD Controller extracts the URL, retrieves the referenced MUD file, and verifies its authenticity. Upon verification, it retrieves the curve name EC with the corresponding parameters G , n , and the key commitment X_c . It then samples a random challenge C and includes it in option 161 of the DHCP *Offer* message.
- 3) The IoT device computes the ZKP response as $Z = r + H \cdot K_c$, where $H = \text{Hash}(R \parallel X_c \parallel C \parallel \text{URL})$, and $\text{Hash}(\cdot)$ is a secure hash function [51]. The device includes Z in option 161 of the DHCP *Request* message. In parallel, the MUD Controller computes the expected hash value H as well.
- 4) The MUD Controller extracts the received ZKP response, and verifies the equality $Z \cdot G \stackrel{?}{=} R + H \cdot X_c$. If the check passes, the *MUD-Binding* verification is successful. In such a case, the MUD Controller responds with a DHCP *Ack* message, translates the ACLs in the MUD file into network policies, and enforces them. Otherwise, the procedure is aborted and network access is denied.

Upon device authentication, the MUD Controller saves a table, termed as *MUD table*, with entries the device MAC address, its MUD_{ID} , the received MUD URL, and the MUD file signature.

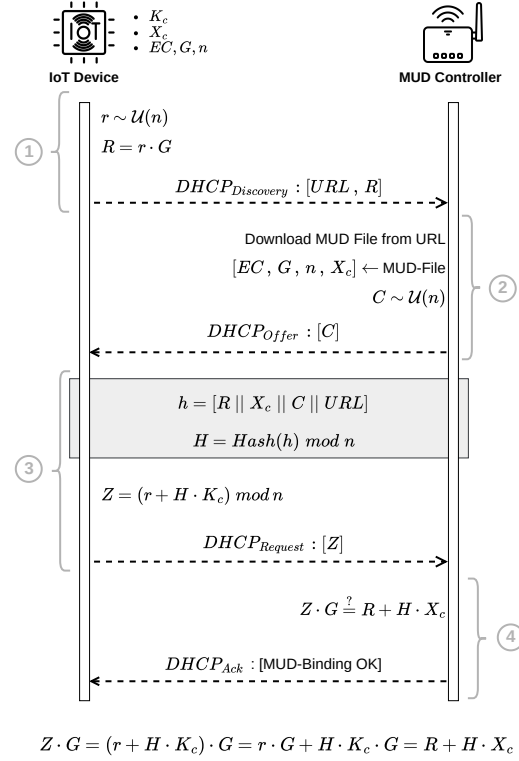
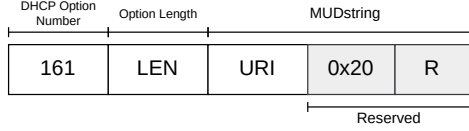


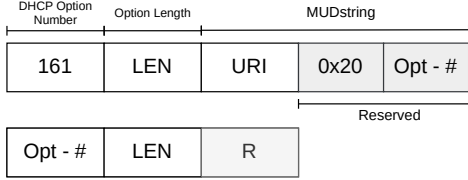
Figure 3: FIDEM ZKP-based MUD Binding verification procedure avoiding manufacturers participation.

5.4.1. ZKP Parameters Embedding. As backward compatibility is one of our priorities and constraints, it is essential to preserve message formats as defined by the standard. To achieve this, we leverage the available reserved field in the MUDstring to carry the commitment R as illustrated in Fig. 4a [10]. However, the space available in the reserved field may not suffice to include the full EC point. In fact, DHCP messages impose strict size limitations: each option can carry at most 255 bytes, and the entire DHCP message cannot exceed 576 bytes [52]. These constraints can make it challenging to embed all required data within a single option. To address this limitation, we propose an alternative approach: include the commitment R in a separate available DHCP option and specify the option number used in the reserved field, as shown in Fig. 4b. This design ensures interoperability with devices from different vendors, which may already use custom DHCP options for other purposes. By providing flexibility in option allocation, we avoid collisions and maintain compatibility across heterogeneous environments.

5.4.2. MUD Profile Update. Maintaining correct and updated network restrictions on MUD-enabled IoT networks requires a secure MUD profile update mechanism. To address potential changes in MUD files due to corrections, FIDEM leverages a publish-subscribe approach [13]. The MUD Controller periodically verifies whether the MUD file signatures of connected devices differ from those stored in



(a) Embedding of commitment into the reserved space of the MUDstring.



(b) Embedding R into an available option. The reserved string contains the number of the options used.

Figure 4: FIDEM ZKP parameters embedding in *DHCP Discovery* message.

the MUD table. If a mismatch is detected, the Controller automatically retrieves the updated MUD file, refreshes its signature, and enforces the new network restrictions. Conversely, when an IoT device already registered in the MUD table issues a new URL, the *MUD-Binding* verification procedure must be executed again. In this case, an additional check is required compared to the initial registration. We observe that a device cannot change its manufacturer during its lifecycle [17]. Therefore, if the new URL points to a MUD file associated with a different manufacturer, the MUD Controller aborts the procedure, and if possible, alerts the network administrator of suspicious behavior. In scenarios where a software update does not alter the device profile, the device typically reboots to apply the update. This reboot triggers a new DHCP handshake. Since the profile remains unchanged, the URL matches the entry stored in the MUD table. In this specific case, the Controller can skip the verification procedure, and omit the challenge C in the DHCP *Offer* message. Appendix C provides a flowgraph summarizing the update procedure described above.

6. Security Analysis

This section presents the security analysis of the *MUD-Binding* verification procedure proposed in FIDEM explaining how attacker objectives **O1–O3** are mitigated.

6.1. Security Considerations

Security Foundations. The security of the *MUD-Binding* verification mechanism presented in Sec. 5.4 relies on two fundamental considerations. First, each device stores the private key K_c in tamper-resistant memory, making physical extraction infeasible, in line with assumptions in prior works [26], [41], [43], [44]. Moreover, recovering K_c from X_c is computationally infeasible under the Elliptic

Curve Discrete Logarithm (ECDL) assumption [53]. The same holds for the nonce r with respect to the commitment R . Under these assumptions, an adversary observing the pair (R, Z) , and knowing the EC parameters cannot recover K_c or r . Consequently, a malicious IoT device unaware of the correct K_c cannot compute a valid response to the challenge of the MUD Controller. Since the K_c and r are never transmitted, these properties address the attacker objective **O1** (secret key recovery).

Manufacturer and Profile Integrity. FIDEM enforces manufacturer and profile integrity through two mechanisms. During initial verification, the MUD Controller validates the signature of the MUD file retrieved at the supplied URL, ensuring authenticity of the device manufacturer. During profile updates, FIDEM guarantees that a device cannot change manufacturer over its lifetime. If a newly-issued MUD URL points to a file signed by a different manufacturer, the MUD Controller aborts the protocol and flags the event as suspicious. This prevents compromised or counterfeit devices from migrating across manufacturers or issuing unauthorized URLs. Such properties address the attacker objective **O2** (cross-manufacturer impersonation).

Resistance to Replay Attacks and Oracle Abuse. The attacker may record legitimate (R', Z') pairs and replay them to bypass verification, luring the MUD Controller to enforce incorrect restrictions. Alternatively, the attacker may leverage R_{oracle} as a cryptographic oracle to compute ZKP values on behalf of the compromised device D_1 . To prevent such attacks, FIDEM relies on an interactive Schnorr protocol in which each execution uses a fresh challenge C . Since a previously recorded ZKP response Z' is bound to an old challenge C' sampled by the MUD Controller, it cannot be replayed in a new session. In fact, $H' = \text{Hash}(R \parallel X_c \parallel C' \parallel \text{URL})$ used to compute Z' , differs from $H = \text{Hash}(R \parallel X_c \parallel C \parallel \text{URL})$ since $C \neq C'$. Furthermore, the *MUD-Binding* verification occurs during the DHCP handshake, before the device obtains network access. Therefore, a compromised device D_1 cannot communicate with R_{oracle} during the verification phase. These properties eliminate adaptive oracle attacks and address the attacker objective **O3** (MUD profile escalation).

Security Under Nonce Reuse Scenarios. Nonce uniqueness is critical for interactive ZKP-based security. If the same r is reused across two sessions with different challenges, an adversary can eliminate it and compute $K_c = (Z_1 - Z_2)(H_1 - H_2)^{-1} \mod n$, breaking the binding mechanism. Therefore, the MUD Controller must track past commitments R to detect reuse. Conversely, using different nonces with the same challenge does not compromise security, as H incorporates R , ensuring unlinkability.

In summary, FIDEM addresses all security concerns raised by the threat model and ensures that the MUD Controller can securely prove *MUD-Binding* even in the presence of compromised or counterfeit devices.


```

Verification summary:

(Q1) - Query not attacker(Kc[]) is true.

(Q2) - Query not attacker(r[]) is true.

(Q3) - Query inj-event(successful_binding(true))
      ==> inj-event(tx_commitment(R_2)) is true.

(Q4) - Query inj-event(successful_binding(true))
      ==> inj-event(tx_challenge(C_2)) is true.

(Q5) - Query inj-event(successful_binding(true))
      ==> inj-event(tx_response(Z_2)) is true.

```

Figure 5: Output for FIDEM formal verification with ProVerif. All queries are validated, proving FIDEM security.

6.2. Security Assessment with ProVerif

To rigorously validate the security of the proposed ZKP-based binding mechanism (Sec. 5.4), we use ProVerif [54], a widely recognized tool for automated protocol analysis [55]–[58]. ProVerif is especially useful when creating new protocols that leverage existing cryptographic primitives, whose security has already been proven, as is the case for FIDEM. ProVerif evaluates the security of the protocol under the Dolev-Yao attacker model [56]. If an attack is possible, ProVerif provides a trace to reproduce it. We formally modeled FIDEM in a single protocol round and executed multiple queries to verify secrecy, resistance to guessing attacks, and authentication properties. Interested readers can find the source code of FIDEM within ProVerif in our GitHub repository. Fig. 5 summarizes the results of our formal security analysis. Appendix D provides detailed query explanation and interpretation. Queries **Q1** and **Q2** prove that secrets r and Kc are not exposed to the adversary. They represent the secret nonce r and the MUD class key Kc , respectively. Successful queries **Q3**, **Q4**, **Q5** confirm injective correspondence between successful bindings, commitments R , challenges C , and responses Z , respectively, thereby preventing replay attacks and ensuring freshness. In summary, our automated analysis through ProVerif shows that FIDEM provides strong secrecy and *MUD-Binding* guarantees.

7. Experimental Evaluation

This section presents our extensive experimental evaluation of FIDEM. Our PoC source code is available on GitHub, while implementation details are in Appendix E.

7.1. Experimental Setup and Settings

Setup. Fig. 6 illustrates the setup of our experimental PoC implementation of FIDEM. We use two reference IoT devices. We first consider the ESP32-S3, an embedded resource-constrained IoT device equipped with a dual-core XTensa LX7 MCU, 512 KB of internal SRAM, and integrated with a 2.4 GHz, 802.11 b/g/n WiFi-4 card.² This

2. <https://www.espressif.com/en/products/socs/esp32-s3>

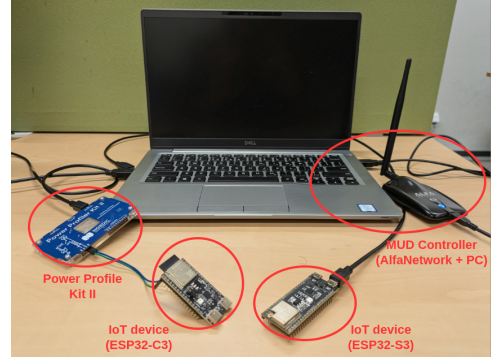


Figure 6: FIDEM PoC implementation setup.

device does not feature secure storage capabilities, allowing us to deploy and test our solution on a general-purpose hardware not equipped with advanced security features. Moreover, we also consider the ESP32-C6 microcontroller, equipped with a 32-bit RISC-V processor, a 320 KB ROM and a 512 KB SRAM, and integrated with a 2.4 GHz, 802.11 b/g/n WiFi-6 card.³ It also features a hardware Trusted Execution Environment (TEE), providing *secure storage* and protected cryptographic operations. Including such a device in our PoC and testing performance on it allows us to evaluate the impact on performance of a hardware capable of enforcing protection of cryptographic secrets stored onboard the device. We implement the MUD Controller using a DELL Latitude 7400 laptop equipped with Ubuntu 22.04.5 LTS and an AlfaNetwork antenna.⁴ We collect energy consumption using a Power Profiler Kit II from Nordic,⁵ a high-precision measurement tool designed to monitor real-time current draw and energy usage of embedded devices. It connects directly to the IoT device and provides accurate profiling of power consumption during different operational phases. For implementation, we use the standard EC *secp256r1* [59].

Settings. We experimentally evaluate FIDEM with respect to two key metrics: (i) computational overhead and (ii) additional energy consumption induced by the ZKP-based *MUD-Binding* verification procedure. For comparison, we consider three benchmarks: (i) *standard DHCP*, following the standard RFC8520 [10]; (ii) *X.509 with HTTP*, using the standard X.509 certificate extension, shared via HTTP payload; and (iii) *X.509 with TLS*, using the standard X.509 certificate extension, used for TLS handshake [60]. Where possible, we implement FIDEM with and without using the TEE, to assess the overhead due to TEE involvement and computations. For each scenario, we collect data from 100 protocol runs and report results through boxplots, showing the median value of the metric of interest. We assess the computational overhead by measuring the time required to carry out the *MUD-Binding* verification procedure

3. <https://www.espressif.com/en/products/socs/esp32-c6>

4. https://www.alfa.com.tw/products/awus036nha?_pos=6&_sid=569e0ca77&_ss=r&variant=36473966166088

5. nordicsemi.com/startppk2

presented in Sec. 5.4. Therefore, we consider the elapsed time of the overall *MUD-Binding* verification procedure at MUD Controller side, from the reception of the DHCP *Discovery* message, up to reception and verification of the ZKP response received. Note that this is the overall time to complete the procedure, including also operations executed on the IoT device under test. We evaluate energy consumption only on the IoT device, as the MUD Controller is not resource-constrained and its energy usage is considered not to be a concern. We consider the energy consumed starting from system setup (including cryptographic context initialization) up to the reception of the DHCP *Ack* message, signaling successful *MUD-Binding*.

7.2. Results

Figure 7 reports our complete experimental results in terms of computational and energy overhead for *MUD-Binding* verification using FIDEM with ESP32-S3 (Fig. 7a, 7c) and ESP32-C6 (Fig. 7b, 7d) devices.

Considering the execution time with the ESP32-S3 device (Fig. 7a), FIDEM introduces minimal delay compared to the standard DHCP method. FIDEM requires a median time of $9.06ms$, which is $\sim 30\%$ more than the time required by the standard DHCP issuing method ($6.99ms$). We recall that, although lightweight, the standard DHCP mechanism does not ensure *MUD-Binding*, being thus unsuitable for secure deployments. At the same time, FIDEM significantly outperforms X.509 leveraging both HTTP and TLS, which require a median time of $1.05s$ and $1.79s$, respectively. Such a large time difference is due to the additional operations required by connection establishment. X.509 with HTTP requires setting up a TCP connection, and X.509 with TLS adds several further cryptographic PKI-based operations during the handshake to set up a secure connection. We acknowledge that such a secure connection provides many more usable security guarantees beyond secure device enrollment and *MUD-Binding*; however, it requires a PKI, expensive to deploy and manage for IoT manufacturers. As demonstrated by our proposed FIDEM approach, this is not necessary to ensure *MUD-Binding*.

When considering the energy consumption with the ESP32-S3 (Fig. 7c), we notice that standard DHCP method exhibits the lowest energy usage, i.e., a median of $354mJ$. We can explain this result by recalling that no security checks are performed when using this mode. Our proposed solution, FIDEM, introduces a slight increase compared to DHCP, i.e., a median consumption of $385mJ$, for an overhead of $\sim 8\%$. This negligible energy overhead demonstrates that EC-based ZKP operations are suitable for constrained devices. X.509 with HTTP shows an energy consumption comparable to FIDEM i.e. a median of $370mJ$. Here, we notice an apparent mismatch between the relationship of time and energy consumption between X.509 with HTTP and FIDEM: although FIDEM is quicker, it consumes approximately the same energy. We can explain these results by recalling that FIDEM relies on cryptographic operations executed in the Cryptographic Acceleration and

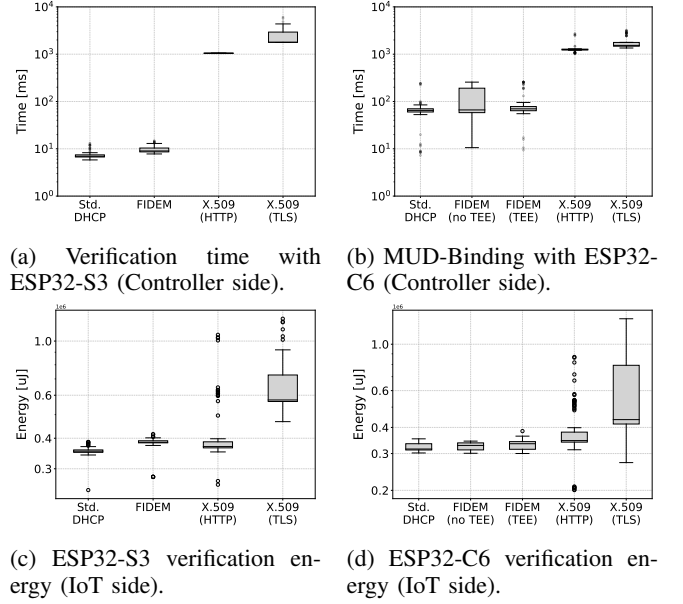


Figure 7: MUD-Binding verification time and energy using FIDEM.

Assurance Modules (CAAM) module of the ESP-32 device. Activating and using such dedicated hardware requires additional energy, partly outweighing the energy gained from the reduced duration of the protocol compared to X.509 over HTTP. Moreover, we recall that using HTTP to issue the X.509 certificate lacks security guarantees: the certificate is transmitted as a plain payload and can be replayed, failing to ensure the binding property (R3, see Sec. 5.1). Conversely, X.509 over TLS incurs a substantial energy overhead, i.e., a median of $574mJ$, nearly double that of FIDEM, with wide variance due to the complexity of the TLS handshake. This high consumption is due to the time required to complete message exchange (as noticed for Fig. 7a) and due to the higher number of cryptography operations involved. Furthermore, during the tests, we observed several network failures, which have an additional influence on the time required to complete the *MUD-Binding* process.

When experimentally evaluating FIDEM on the ESP32-C6 device, we observe that the performance trend follows the one already discussed for the ESP32-S3 device: FIDEM is $\times 20$ faster and consumes $\sim 35\%$ less energy compared to X.509 over TLS. At the same time, compared to the trends observed for the ESP32-S3 device, we notice that using the ESP32-C6 device increases the baseline time required for DHCP handshake. This is evident when comparing the standard DHCP data series between Fig. 7a and Fig. 7b. This is due to several factors, including differences in architecture, hardware, and wifi cards between the two models, which lead to a larger fixed connection plus stack cost. Nevertheless, the ESP32-C6 device is natively optimized for cryptographic operations, allowing to reduce FIDEM time overhead compared to standard DHCP to $\sim 7\%$ ($\sim 5ms$). Furthermore, we also compare the performance of FIDEM

on the ESP32-C6 device with and without using the TEE secure hardware, to evaluate the additional delay and energy consumption introduced by such a component. Fig. 7b shows that leveraging the TEE introduces limited computational delay of $\sim 3.5ms$ and $\sim 6mJ$ increased energy. This overhead is induced by the execution of required operations for switching from the insecure to the secure worlds. Finally, we notice that although taking longer execution times, the ESP32-C6 requires less energy consumption compared to the ESP32-S3. Specifically, FIDEM (with TEE) requires $\sim 334mJ$. This indicates a lower average power profile and design of the ESP32-C6, which favors idle states during waiting periods.

In summary, our results demonstrate that FIDEM achieves strong security guarantees with negligible impact on the resources and lifetime of constrained IoT devices.

8. Discussion

Out-of-Band Communications. Aligned with the MUD standard RFC8520 [10], our system model considers only IoT traffic traversing a monitoring entity (e.g., a router), where MUD profiles are enforced. Accordingly, out-of-band channels such as cellular networks or direct device-to-device communications (e.g., Bluetooth or ZigBee) are excluded. While resource-constrained devices rarely support cellular connectivity, short-range communication technologies are commonly available in higher-end IoT devices. These channels could be exploited by an attacker to bypass the router and offload cryptographic operations to D_{oracle} , thereby undermining *MUD-Binding* verification. However, addressing such scenarios would require extending MUD beyond IP-based communications, entailing a fundamentally different architectural model. This would significantly alter both the system and threat models assumptions considered in this work, effectively defining a distinct research problem rather than a direct extension of FIDEM. Investigating native MUD integration into non-IP protocol stacks therefore represents a promising direction for future work.

Scalability. FIDEM operates exclusively during the DHCP handshake, i.e., before the device receives network connectivity and before any operational traffic is allowed. As such, it does not introduce persistent computational or communication overhead during normal device operation. In contrast, the only additional cost is a one-time, bounded delay during onboarding ($\sim 5ms$), as quantified in our evaluation. Consequently, scalability in this context is related to the MUD Controller’s capability to handle multiple concurrent onboarding requests. However, this is a property inherent to the implementation and deployment of the Controller, not to FIDEM itself. Similarly, potential Denial-of-Service (DoS) scenarios target the Controller’s exposure during the DHCP phase, which already exists in standard deployments. Thus, FIDEM does not amplify this attack surface beyond a minimal and short-lived processing overhead.

Extension to Non-Interactive Scenarios. Our design of FIDEM focuses on the DHCP extension due to its

widespread adoption for IP address assignment and its support in current MUD implementations. The MUD standard also defines an LLDP extension for URL issuance; however, LLDP is inherently non-interactive. As a result, FIDEM cannot be directly extended using non-interactive Schnorr-based ZKP authentication [61], as this would expose the system to pre-computation and replay attacks. A possible mitigation is to leverage interaction with the *Operating Server* for device identity authentication. However, as discussed, device authentication alone does not guarantee *MUD-Binding*: a malicious device may successfully authenticate while still issuing a spoofed URL. To address this, after successful authentication, the Controller can rely on the verified device identity to directly retrieve the corresponding MUD file from the MUD Server, rather than trusting the URL advertised by the device. While this approach involves the MUD Server during file retrieval, it does not require additional cryptographic operations.

MUD Management Model and Group-Based Authentication. Our proposed *MMM* associates a single secret key (K_c) with each MUD class. While we assume secure storage of cryptographic material on IoT devices, this design introduces an inherent limitation: if K_c is compromised, all devices within the corresponding MUD class are affected. A potential mitigation, while still avoiding reliance on a PKI and digital certificates, is the adoption of group-based authentication schemes, where devices share a common public key but hold distinct private keys [62]. In this setting, a class-level public key could be included in the MUD file, while each device is provided with a unique private key, thus overcoming the shared-secret limitation of our *MMM*. However, group signature schemes introduce additional challenges, including complex key revocation, scalability limitations, and dependence on trusted setup procedures [62]–[64]. Considering such limitations and our system model, we believe that the proposed ZKP-based approach at the roots of FIDEM offers the best compromise at present. These trade-offs suggest that alternative *MMMs* require careful evaluation, motivating future research toward designs that better balance security, scalability, and efficiency.

9. Conclusions

In this paper, we presented FIDEM, a framework designed to address the fundamental gap in the MUD standard by securing the widely adopted but insecure DHCP-based MUD URL issuance mechanism. FIDEM leverages Schnorr-based ZKP authentication to cryptographically enforce the *MUD-Binding* property while remaining fully standard-compliant and backward compatible. By design, FIDEM withstands a stronger adversarial model than existing approaches, including attackers capable of exploiting legitimate devices as cryptographic oracles. Furthermore, it achieves this without relying on PKI, avoiding the associated operational complexity, and without requiring active manufacturer involvement, thus preserving scalability and deployability in heterogeneous IoT environments.

Experimental results from a real-world PoC demonstrate that these security guarantees are achieved with minimal overhead compared to standard DHCP, while significantly outperforming certificate-based solutions ($\times 20$ faster and $\sim 35\%$ less energy than X.509 over TLS). Future work will investigate extensions to LLDP-based deployments, mitigation of out-of-band communication threats, and alternative strategies such as group-based digital signatures.

References

- [1] M. Lezzi, M. Lazoi, and A. Corallo, "Cybersecurity for industry 4.0 in the current literature: A reference framework," *Computers in Industry*, vol. 103, pp. 97–110, 2018. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0166361518303658>
- [2] G. Culot, F. Fattori, M. Podrecca, and M. Sartor, "Addressing industry 4.0 cybersecurity challenges," *IEEE Engineering Management Review*, vol. 47, no. 3, pp. 79–86, 2019.
- [3] P. Kumari and A. K. Jain, "A comprehensive study of ddos attacks over iot network and their countermeasures," *Computers & Security*, vol. 127, p. 103096, 2023. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167404823000068>
- [4] S. Dhanda, B. Singh, and P. Jindal, "Lightweight cryptography: A solution to secure iot," *Wireless Personal Communications*, vol. 112, pp. 1947 – 1980, 2020.
- [5] V. A. Thakor, M. A. Razzaque, and M. R. A. Khandaker, "Lightweight cryptography algorithms for resource-constrained iot devices: A review, comparison and research opportunities," *IEEE Access*, vol. 9, pp. 28 177–28 193, 2021.
- [6] A. E. Adeniyi, R. G. Jimoh, and J. B. Awotunde, "A systematic review on elliptic curve cryptography algorithm for internet of things: Categorization, application areas, and security," *Computers and Electrical Engineering*, vol. 118, p. 109330, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0045790624002581>
- [7] F. Meneghello, M. Calore, D. Zucchetto, M. Polese, and A. Zanella, "Iot: Internet of threats? a survey of practical security vulnerabilities in real iot devices," *IEEE Internet of Things Journal*, vol. 6, no. 5, pp. 8182–8201, 2019.
- [8] L. Houben, T. Terhoeve, and S. Sciancalepore, "Mudthread: Securing constrained iot networks via manufacturer usage descriptions," *IEEE Communications Magazine*, vol. 63, no. 2, pp. 128–134, 2025.
- [9] C. Kolias, G. Kambourakis, A. Stavrou, and J. Voas, "Ddos in the iot: Mirai and other botnets," *Computer*, vol. 50, no. 7, pp. 80–84, 2017.
- [10] E. Lear, R. Droms, and D. Romascanu, "Manufacturer Usage Description Specification," RFC 8520, Mar. 2019. [Online]. Available: <https://www.rfc-editor.org/info/rfc8520>
- [11] J. L. Hernández-Ramos, S. N. Matheu, A. Feraudo, G. Baldini, J. B. Bernabe, P. Yadav, A. Skarmeta, and P. Bellavista, "Defining the behavior of iot devices through the mud standard: Review, challenges, and research directions," *IEEE Access*, vol. 9, pp. 126 265–126 285, 2021.
- [12] N. Mazhar, R. Salleh, M. Zeeshan, and M. M. Hameed, "Role of device identification and manufacturer usage description in iot security: A survey," *IEEE Access*, vol. 9, pp. 41 757–41 786, 2021.
- [13] A. Feraudo, P. Yadav, R. Mortier, P. Bellavista, and J. Crowcroft, "Sok: Beyond iot mud deployments—challenges and future directions," *arXiv preprint arXiv:2004.08003*, 2020.
- [14] L. Morgese Zangrandi, T. Van Ede, T. Booiij, S. Sciancalepore, L. Allodi, and A. Continella, "Stepping out of the mud: Contextual threat information for iot devices with manufacturer-provided behavior profiles," in *Proceedings of the 38th Annual Computer Security Applications Conference*, 2022, pp. 467–480.
- [15] F. Corno and L. Mannella, "A gateway-based mud architecture to enhance smart home security," in *2023 8th International Conference on Smart and Sustainable Technologies (SpliTech)*, 2023, pp. 1–6.
- [16] S. N. M. García, A. Molina Zarca, J. L. Hernández-Ramos, J. B. Bernabé, and A. S. Gómez, "Enforcing behavioral profiles through software-defined networks in the industrial internet of things," *Applied Sciences*, vol. 9, no. 21, 2019. [Online]. Available: <https://www.mdpi.com/2076-3417/9/21/4576>

- [17] M. Richardson, W. Pan, and E. Lear, "Authorized update to mud urls," Internet Engineering Task Force, Internet-Draft draft-ietf-iotops-mud-acceptable-urls-01, nov 2025, work in Progress. [Online]. Available: <https://datatracker.ietf.org/doc/draft-ietf-iotops-mud-acceptable-urls/01/>
- [18] S. Rashmikan, W. Brian, M. Cheryl, and L. Eliot, "Mud-manager." [Online]. Available: <https://github.com/CiscoDevNet/MUD-Manager>
- [19] D. Dodson, T. Polk, M. Souppaya, W. C. Barker, P. Grayeli, M. Raguso, and S. Symington, "Securing small-business and home internet of things (iot) devices," 2019.
- [20] "Open source mud manager (osmud)," 2018. [Online]. Available: <https://osmud.org/>
- [21] M. Ranganathan, D. Montgomery, and O. E. Mimouni, "Soft mud: Implementing manufacturer usage descriptions on openflow sdn switches." ThinkMind, Valencia, ES, 2019-03-24 04:03:00 2019. [Online]. Available: https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=927289
- [22] S. M. Sajjad, M. Yousaf, H. Afzal, and M. R. Mufti, "emud: Enhanced manufacturer usage description for iot botnets prevention on home wifi routers," *IEEE Access*, vol. 8, pp. 164 200–164 213, 2020.
- [23] R. T. Fielding and J. Reschke, "Hypertext transfer protocol (http/1.1): Message syntax and routing," RFC 7230, jun 2014. [Online]. Available: <https://www.rfc-editor.org/info/rfc7230>
- [24] T. Polk, M. Souppaya, and W. C. Barker, "Mitigating iot-based automated distributed threats," *NIST, Gaithersburg, MD, USA*, 2017. [Online]. Available: <https://www.nccoe.nist.gov/sites/default/files/legacy-files/iot-ddos-project-description-draft.pdf>
- [25] L. Chang, "A proactive approach to detect iot based flooding attacks by using software defined networks and manufacturer usage descriptions," Master's thesis, Arizona State University, 2018.
- [26] S. Datta, V. U, and M. R. K, "Mudscan: Double authentication based secure control mechanism for mud enable iot networks," *Peer-to-Peer Networking and Applications*, vol. 18, no. 3, p. 139, 2025.
- [27] D. R. George, W. van Hoof, H. Mostafaei, and S. Sciancalepore, "MeshGuard - MUD-Based Network Access Control for Large-Scale Thread-Powered IoT Networks," in *56th Annual IEEE/IFIP International Conference on Dependable Systems and Networks*. IEEE, 2026.
- [28] S. N. Matheu, A. Robles Enciso, A. Molina Zarca, D. Garcia-Carrillo, J. L. Hernández-Ramos, J. Bernal Bernabe, and A. F. Skarmeta, "Security architecture for defining and enforcing security profiles in dlt/sdn-based iot systems," *Sensors*, vol. 20, no. 7, 2020. [Online]. Available: <https://www.mdpi.com/1424-8220/20/7/1882>
- [29] J. Jimenez, "Using mud on coap environments," Internet Engineering Task Force, Internet-Draft draft-jimenez-t2trg-mud-coap-00, mar 2020, work in Progress. [Online]. Available: <https://datatracker.ietf.org/doc/draft-jimenez-t2trg-mud-coap/00/>
- [30] M. Richardson, J. Latour, and H. H. Gharakheili, "Loading manufacturer usage description (mud) urls from qr codes," RFC 9238, may 2022. [Online]. Available: <https://www.rfc-editor.org/info/rfc9238>
- [31] S. Sciancalepore, M. Vučinić, G. Piro, G. Boggia, and T. Watteyne, "Link-layer security in TSCH networks: effect on slot duration," *Transactions on emerging telecommunications technologies*, vol. 28, no. 1, p. e3089, 2017.
- [32] F. Forsby, M. Furuheid, P. Papadimitratos, and S. Raza, "Lightweight x.509 digital certificates for the internet of things," in *Interoperability, Safety and Security in IoT*, G. Fortino, C. E. Palau, A. Guerrieri, N. Cuppens, F. Cuppens, H. Chaouchi, and A. Gabillon, Eds. Springer International Publishing, 2018, pp. 123–133.
- [33] J. P. Mattsson, G. Selander, S. Raza, J. Höglund, and M. Furuheid, "Cbor encoded x.509 certificates (c509 certificates)," Internet Engineering Task Force, Internet-Draft draft-ietf-cose-cbor-encoded-cert-15, aug 2025, work in Progress. [Online]. Available: <https://datatracker.ietf.org/doc/draft-ietf-cose-cbor-encoded-cert/15/>
- [34] A. M. Mandalari, H. Haddadi, D. J. Dubois, and D. Choffnes, "Protected or porous: A comparative analysis of threat detection capability of iot safeguards," in *2023 IEEE Symposium on Security and Privacy (SP)*, 2023, pp. 3061–3078.
- [35] B. Kuang, A. Fu, W. Susilo, S. Yu, and Y. Gao, "A survey of remote attestation in internet of things: Attacks, countermeasures, and prospects," *Computers & Security*, vol. 112, p. 102498, 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167404821003229>
- [36] M. Ambrosin, M. Conti, R. Lazzeretti, M. M. Rabbani, and S. Ranise, "Collective remote attestation at the internet of things scale: State-of-the-art and future challenges," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 4, pp. 2447–2461, 2020.
- [37] I. De Oliveira Nunes, S. Jakkamsetti, N. Rattanavipanon, and G. Tsudik, "On the toctou problem in remote attestation," in *Proceedings of the 2021 ACM SIGSAC Conference on Computer and Communications Security*, ser. CCS '21. New York, NY, USA: Association for Computing Machinery, 2021, p. 2921–2936. [Online]. Available: <https://doi.org/10.1145/3460120.3484532>
- [38] D. H. Yum and P. J. Lee, "Generic construction of certificateless encryption," in *Computational Science and Its Applications – ICCSA 2004*, A. Laganá, M. L. Gavrilova, V. Kumar, Y. Mun, C. J. K. Tan, and O. Gervasi, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2004, pp. 802–811.
- [39] A. Karati, C.-I. Fan, and E.-S. Zhuang, "Reliable data sharing by certificateless encryption supporting keyword search against vulnerable kgc in industrial internet of things," *IEEE Transactions on Industrial Informatics*, vol. 18, no. 6, pp. 3661–3669, 2022.
- [40] A. W. Dent, "A survey of certificateless encryption schemes and security models," *International Journal of Information Security*, vol. 7, no. 5, pp. 349–377, 2008.
- [41] A. Cirne, P. R. Sousa, J. S. Resende, and L. Antunes, "Hardware security for internet of things identity assurance," *IEEE Communications Surveys & Tutorials*, vol. 26, no. 2, pp. 1041–1079, 2024.
- [42] K. Yang, D. Blaauw, and D. Sylvester, "Hardware designs for security in ultra-low-power iot systems: An overview and survey," *IEEE Micro*, vol. 37, no. 6, pp. 72–89, 2017.
- [43] A. Kanuparthi, R. Karri, and S. Addepalli, "Hardware and embedded security in the context of internet of things," in *Proceedings of the 2013 ACM Workshop on Security, Privacy & Dependability for Cyber Vehicles*, ser. CyCAR '13. New York, NY, USA: Association for Computing Machinery, 2013, p. 61–64. [Online]. Available: <https://doi.org/10.1145/2517968.2517976>
- [44] B. Pearson, L. Luo, Y. Zhang, R. Dey, Z. Ling, M. Bassiouni, and X. Fu, "On misconception of hardware and cost in iot security and privacy," in *ICC 2019 - 2019 IEEE International Conference on Communications (ICC)*, 2019, pp. 1–7.
- [45] U.S. Department of Justice, "Leader of massive scheme to traffic fraudulent and counterfeit cisco networking equipment pleads guilty," 2022, last Accessed: 2026-04-29. [Online]. Available: <https://www.justice.gov/archives/opa/pr/leader-massive-scheme-traffic-fraudulent-and-counterfeit-cisco-networking-equipment>
- [46] Kaspersky, "Trojan in fake smartphones," 2024, last Accessed: 2026-04-29. [Online]. Available: <https://www.kaspersky.com/blog/trojan-in-fake-smartphones/53331/>
- [47] U.S. Department of Energy, "Fraudulent and counterfeit networking equipment," 2022, last Accessed: 2026-04-29. [Online]. Available: https://www.energy.gov/sites/default/files/2022-10/OE-3%20Fraudulent%20Networking%20Equipment%20FINAL_0.pdf
- [48] M. El-Hajj and P. Beune, "Lightweight public key infrastructure for the internet of things: A systematic literature review," *Journal of Industrial Information Integration*, vol. 41, p. 100670, 2024. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2452414X24001158>

- [49] J. Höglund, S. Lindemer, M. Furuheid, and S. Raza, “Pki4iot: Towards public key infrastructure for the internet of things,” *Computers & Security*, vol. 89, p. 101658, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167404819302019>
- [50] I. S. Association *et al.*, “Guidelines for use of extended unique identifier (eui), organizationally unique identifier (oui), and company id (cid),” *IEEE: Piscataway, NJ, USA*, 2018.
- [51] R. Sobti and G. Geetha, “Cryptographic hash functions: a review,” *International Journal of Computer Science Issues (IJCSI)*, vol. 9, no. 2, p. 461, 2012.
- [52] R. Droms, “Dynamic Host Configuration Protocol,” RFC 2131, Mar. 1997. [Online]. Available: <https://www.rfc-editor.org/info/rfc2131>
- [53] S. D. Galbraith and P. Gaudry, “Recent progress on the elliptic curve discrete logarithm problem,” *Designs, Codes and Cryptography*, vol. 78, no. 1, pp. 51–72, 2016.
- [54] “Proverif: Cryptographic protocol verifier in the formal model.” [Online]. Available: <https://bbblanche.gitlabpages.inria.fr/proverif/>
- [55] D. R. George and S. Sciancalepore, “epptm—enhanced privacy-preserving trajectory matching on autonomous vehicles,” *IEEE Internet of Things Journal*, vol. 12, no. 13, pp. 24 552–24 569, 2025.
- [56] B. Blanchet, “Modeling and verifying security protocols with the applied pi calculus and proverif,” *Foundations and Trends® in Privacy and Security*, vol. 1, no. 1-2, pp. 1–135, 2016. [Online]. Available: <http://dx.doi.org/10.1561/33000000004>
- [57] E. Wisse, P. Tedeschi, S. Sciancalepore, and R. Di Pietro, “A2rid—anonymous direct authentication and remote identification of commercial drones,” *IEEE Internet of Things Journal*, vol. 10, no. 12, pp. 10 587–10 604, 2023.
- [58] P. Tedeschi, S. Sciancalepore, and R. Di Pietro, “Arid: Anonymous remote identification of unmanned aerial vehicles,” in *Proceedings of the 37th Annual Computer Security Applications Conference*, ser. ACSAC ’21. New York, NY, USA: Association for Computing Machinery, 2021, p. 207–218. [Online]. Available: <https://doi.org/10.1145/3485832.3485834>
- [59] “Standard curve database.” [Online]. Available: <https://std.neuromancer.sk/secg/secp256r1>
- [60] E. Rescorla and T. Dierks, “The Transport Layer Security (TLS) Protocol Version 1.2,” RFC 5246, Aug. 2008. [Online]. Available: <https://www.rfc-editor.org/info/rfc5246>
- [61] M. Eckel, D. R. George, B. Grohmann, and C. Krauß, “Remote attestation with constrained disclosure,” in *Proceedings of the 39th Annual Computer Security Applications Conference*, ser. ACSAC ’23. New York, NY, USA: Association for Computing Machinery, 2023, p. 718–731. [Online]. Available: <https://doi.org/10.1145/3627106.3627118>
- [62] M. N. S. Perera, T. Nakamura, M. Hashimoto, H. Yokoyama, C.-M. Cheng, and K. Sakurai, “A survey on group signatures and ring signatures: Traceability vs. anonymity,” *Cryptography*, vol. 6, no. 1, 2022. [Online]. Available: <https://www.mdpi.com/2410-387X/6/1/3>
- [63] J. Bootle, A. Cerulli, P. Chaidos, E. Ghadafi, and J. Groth, “Foundations of fully dynamic group signatures,” in *Applied Cryptography and Network Security*, M. Manulis, A.-R. Sadeghi, and S. Schneider, Eds. Cham: Springer International Publishing, 2016, pp. 117–136.
- [64] C. Esposito, A. Castiglione, F. Palmieri, and A. D. Santis, “Integrity for an event notification within the industrial internet of things by using group signatures,” *IEEE Transactions on Industrial Informatics*, vol. 14, no. 8, pp. 3669–3678, 2018.

Appendix A. Notation

Table 3 summarizes the notation used in this work.

TABLE 3: Notation used in this work.

Notation	Description
M	Manufacturer identifier
C	MUD Class identifier
MUD_{ID}	MUD identifier: $[M, C]$
EC	Elliptic-Curve
G	Base point of the Elliptic-Curve of order n
n	Order of the Elliptic-Curve
K_c	Secret key associated to the MUD Class C
X_c	Public commitment of K_c for the MUD Class C
$Hash(\cdot)$	Secure hash function
H	Hash value output from $Hash(\cdot)$
r	Random high-entropy nonce for ZKP
R	Commitment of random nonce r for ZKP
C	Random challenge issued by the MUD Controller
Z	ZKP response for MUD -Binding verification

Appendix B. Extended MUD File Template

Listing 1: MUD file extension for FIDEM setup

```
"ietf-mud:mud": {
  "mud-version": 1,
  "mud-url": "<URL>",
  "last-update": "<YY-MM-DD>T<HH:MM:SS>Z",
  "cache-validity": <HOURS>,
  "is-supported": true,
  "systeminfo": "<DESCRIPTION>",

  "crypto-fidem:crypto": {
    "mud-identity": {
      "manufacturer-id": "<HEX>",
      "class-id": "<HEX>"
    },
    "ec-params": {
      "curve": "<CURVE-NAME>",
      "p-format": "<UNCOMPRESSED/COMPRESSED>",
      "class-key": "<EC-POINT>"
    },
    "hash-alg": "<sha-256>"
  },

  "from-device-policy": {
    ...
  },
  "to-device-policy": {
    ...
  },

  "ietf-access-control-list:acls": {
    ...
  }
}
```



```
}

```

Listing 1 shows the proposed extended MUD file template including the FIDEM block to include the necessary cryptographic material for Device-to-MUD Profile Binding (*MUD-Binding*) verification.

Appendix C. MUD Profile Update

Fig. 8 shows the flowchart diagram for the MUD profile update logic implemented at the MUD Controller side with FIDEM.

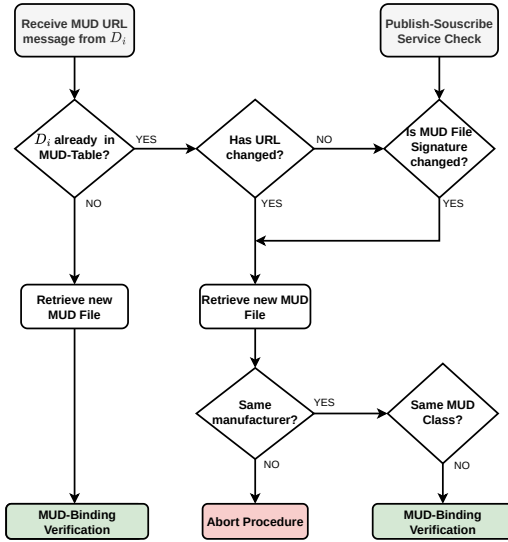


Figure 8: Flow chart of the MUD file update procedure logic implemented in FIDEM.

Appendix D. ProVerif Queries explanation

With reference to the ProVerif queries tested shown in Fig. 5, we provide a detailed description and explanation. The queries output Q1 and Q2 *not attacker(elem[]) is true* indicates that the attacker cannot retrieve the value of *elem[]*. Otherwise, if the output is *false*, the attacker can retrieve it. They represent the secret nonce *r* and the MUD class key *K_c*, respectively. The general query *inj-event(event_A) ==> inj-event(event_B)* expresses an injective correspondence property. It means that for every occurrence of event *event_A*, there is a unique corresponding occurrence of event *event_B* earlier in the protocol execution. If the query output is *true*, every successful *event_A* is guaranteed to correspond to a unique prior *event_B*, thus preventing replay attacks and ensuring that protocol steps are correctly linked. In contrast, *false* output implies the protocol cannot guarantee that every successful *event_A* is tied to a unique prior *event_B*, which undermines correctness and freshness guarantees. In our ProVerif code,

event *successful_binding(true)* represents successful *MUD-Binding* verification, *tx_commitment(R)*, *tx_challenge(C)* and *tx_response(Z)* represent events of sending commitment *R*, challenge *C*, and ZKP response *Z*, respectively. Successful queries Q3, Q4, Q5 confirm injective correspondence between successful bindings, commitments *R*, challenges *C*, and responses *Z*, respectively, thereby preventing replay attacks and ensuring freshness.

Appendix E. PoC Implementation Details

For PoC and overhead analysis purposes, we avoid implementing procedures and communications that remain invariant across different Device-to-MUD Profile Binding (*MUD-Binding*) verification approaches. For example, MUD file retrieval from a remote server is common regardless of the considered approach. Therefore, MUD files are stored locally and authenticated with a locally generated private key, as if it were the manufacturer's secret key. The corresponding public key is certified by a locally generated certificate authority. For key generation and cryptographic operations, we leveraged the OpenSSL API.

MUD Controller Implementation The MUD controller logic runs on an Ubuntu laptop equipped with an Alfa Network AR9271 USB antenna configured as an access point. We leveraged *hostapd* and *systemd-networkd* to control the network interface and manage Access Point (AP) configuration. The AP uses a static IP address (e.g., 10.42.0.1/24), and a Kea DHCPv4 server handles DHCP connections. Kea exposes API hooks for custom logic and DHCP option management. Specifically, we implemented hooks using the *pkt4_receive* and *pkt4_send* callouts to inspect incoming packets and log custom options such as Option 161 (MUD URL). We also leveraged OpenSSL for cryptographic computations and implemented X.509 extensions for MUD URL issuance via HTTP or TLS handshake. To support this, at startup, we launched a dedicated thread listening on the configured port for certificate exchange and URL transmission. We measure the *MUD-Binding* verification time at MUD Controller side using the *clock_gettime(·,·)* as part of the POSIX API. Additional relevant aspects include:

- NetworkManager was configured to ignore the Alfa interface to avoid conflicts with *hostapd* and static IP assignment.
- A wrapper workflow ensures reliable Kea restarts after hot-plug events by cleaning stale PID/lock files and recreating runtime directories.
- ACLs were applied to allow Kea (running as a system user) to write logs under the designated directory.

IoT Device Implementation FIDEM logic was implemented on an ESP32 microcontroller using the Espressif API, which provides hooks to manipulate DHCP packets, similar to the controller. We integrated basic HTTP and

TLS clients for X.509 certificate transmission and sharing the MUD URL with the controller. Cryptographic operations leverage hardware acceleration and an asynchronous worker thread. Upon challenge reception, the worker computes the hash value H and the ZKP response Z while the main process builds the standard DHCP Request message before invoking the TX hook. This design minimizes latency introduced by cryptographic computations.

FIDEM Implementation FIDEM MUD binding verification uses elliptic curve `secp256r1`, supported by OpenSSL. The cryptographic parameters include:

- Nonce r : 256 bits
- Challenge C : 256 bits
- Secret key K : 256 bits

Due to DHCP option length restrictions, FIDEM parameters were embedded using custom DHCP options. Specifically, R and `DevID` were placed in available custom options 224 and 225, as shown in Fig. 4b.